



AMAZON RDS

1

(Relational Database Service)

Need a database for your application?

- ✓ Don't want to install or manage databases?
- ✓ Want high availability, backups, and security?
- ✓ Need to focus on your app, not infrastructure?

AWS RDS

makes it
SIMPLE!

Amazon RDS is a managed service that makes it easy to set up, operate, and scale a relational database in the cloud.

You Focus on



Your Application



RDS



Managed Database

AWS Handles

- ✓ Provisioning
- ⚙ Patching
- 🔄 Backups
- 🔒 Security
- 📊 Monitoring
- 📏 Scaling & HA

Why RDS?

- Fully managed relational databases in the cloud.
- Supports popular database engines.
- High availability and durability built-in.
- Automatic backups and point-in-time recovery.
- Easy scaling for read traffic.
- Pay only for what you use.

Common Use Cases

- ★ Web & mobile applications
- ★ E-commerce platforms
- ★ IoT data storage
- ★ Analytics & reporting
- ★ SaaS applications



With RDS, AWS takes care of the database
so **YOU** can build amazing applications!

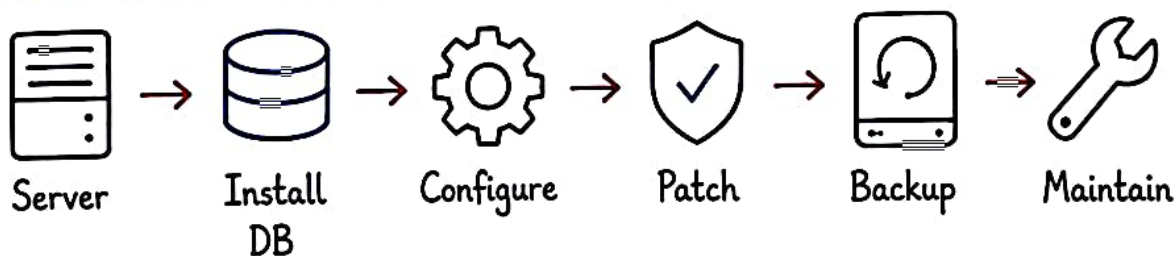


WHAT IS RDS?

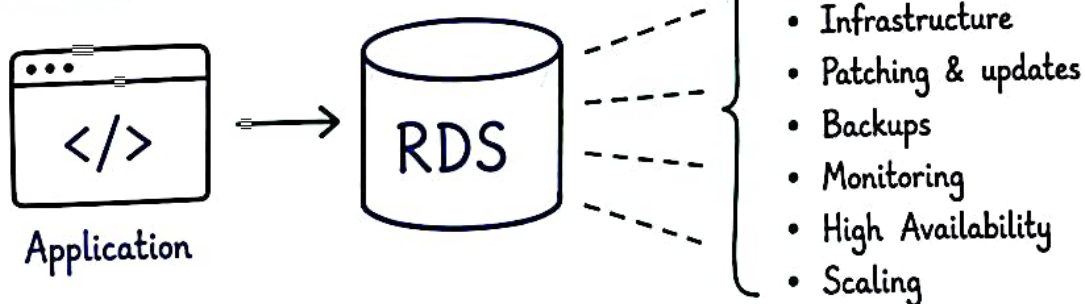
2

Amazon RDS (Relational Database Service) is a managed **AWS** service that makes it easier to set up, operate, and scale relational databases in the cloud.

Traditional Database Management



With Amazon RDS



What "Managed" actually means?

- You don't have to provision or manage database servers.
- Automatic backups and point-in-time recovery.
- Automatic patching of the database engine.
- Built-in monitoring and failure handling.
- Easily scale storage and compute when needed.
- Multi-AZ deployment for high availability.



RDS lets you focus on your application,
not on managing the database.

RDS DATABASE ENGINES

3

RDS supports several popular database engines.

You can choose the one that fits your application.

MySQL



Open source,
widely used,
great for
web apps.

PostgreSQL



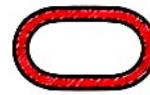
Open source,
powerful,
advanced
features.

MariaDB



MySQL
compatible,
open source,
high
performance.

Oracle



Enterprise
grade,
advanced
features.

SQL Server



Microsoft
ecosystem
friendly.

How to Choose?

- Already using a database? → Pick the same engine in RDS.
- Need open source? → MySQL / PostgreSQL / MariaDB
- Using Microsoft stack? → SQL Server
- Need enterprise features? → Oracle

Choose the engine
based on your
application needs
and team skills.

Key Points

- You can switch engines only by creating a new instance and migrating your data.
- Most engines support Multi-AZ, Read Replicas, Backups, and Automatic Patching.
- RDS handles the heavy lifting so you can focus on your app.



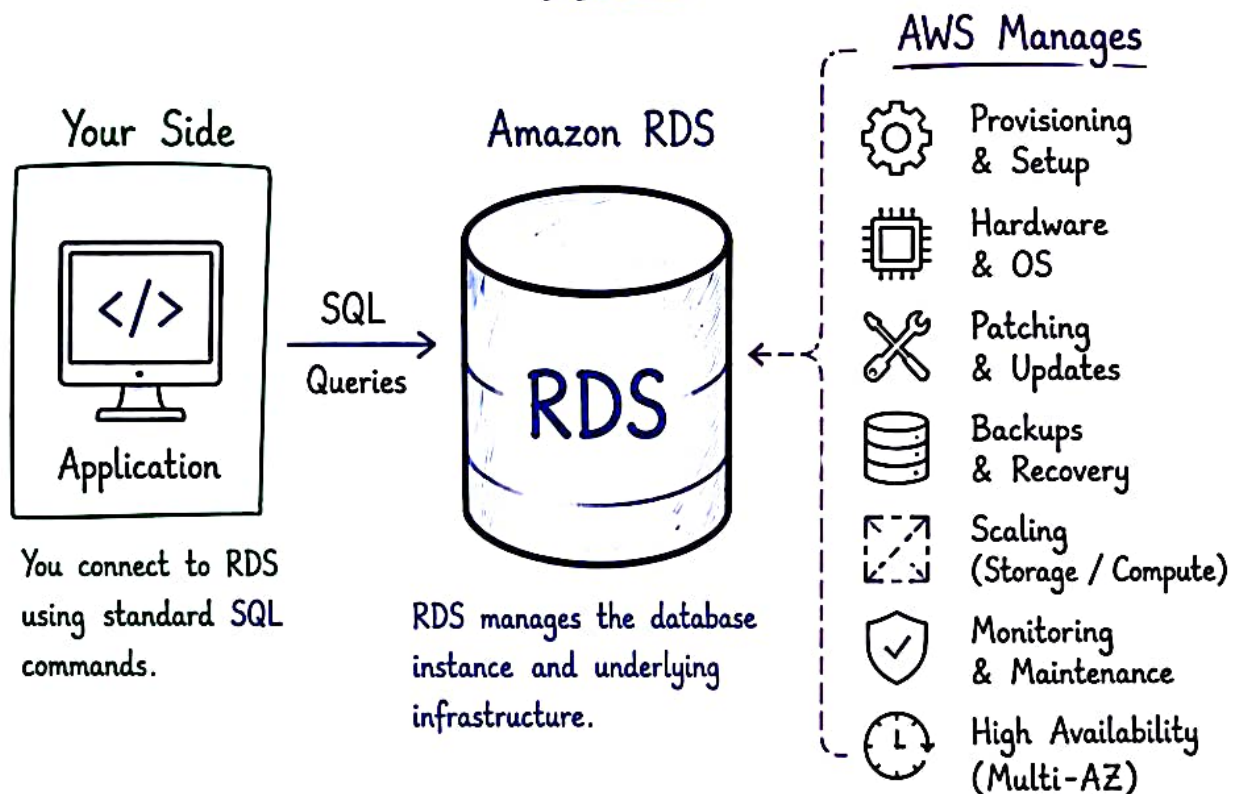
Multiple engines, one managed service.
Freedom to choose, power to build!



How RDS Works

4

RDS takes care of the complex database management tasks so you can focus on your application.



What happens behind the scenes?

- RDS runs your database on a managed infrastructure in AWS.
- It automatically handles updates, backups, and failures.
- You get high availability and durability without managing servers.
- You can easily scale storage or compute as your application grows.



In Simple Words,

You use RDS like a regular database, but **AWS** handles everything hard in the background.

CREATE AN RDS DATABASE

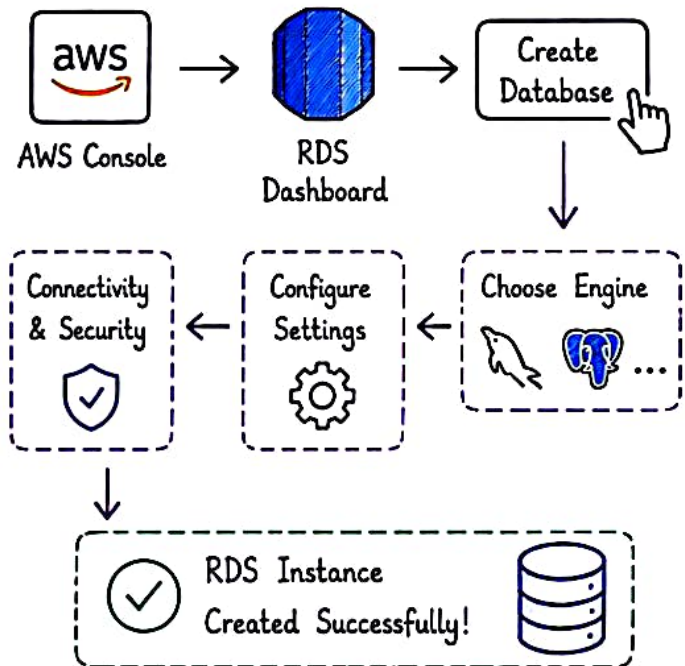
5

Creating an RDS database is simple and takes just a few minutes.

Steps to Create RDS

- 1 Go to RDS Dashboard
- 2 Click on Create Database
- 3 Choose Database Engine (MySQL, PostgreSQL, etc.)
- 4 Select DB Instance Class (t3.micro, t3.small, etc.)
- 5 Configure Settings (DB name, username, password, storage, connectivity)
- 6 Configure Connectivity & Security (VPC, Subnet, Security Group)
- 7 Click Create Database and wait a few minutes.

AWS RDS Console Flow



You will get an endpoint to connect your application.

Two ways to create RDS

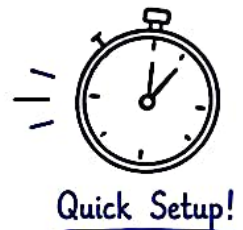
1 Standard Create

- Choose each setting manually.
- More control and flexibility.
- Good for learning and production use.



2 Easy Create

- Minimal configuration.
- Uses default settings.
- Fast and good for testing.



Key Points



- RDS is fully managed, so AWS handles the hard work.
- You just create the database and connect your application.
- You can change instance size, storage, and settings later.
- RDS provides a stable endpoint to connect your app.



RDS INSTANCE & STORAGE

6

In RDS, you choose the right instance type and storage for your database workload.

1. DB INSTANCE (Compute)

The DB instance is the compute part that runs your database engine.

You can choose based on:

- Workload (low, medium, high)
- vCPU and Memory
- Burstable or Dedicated performance
- Cost

Instance Types (Examples)

t Classes (Burstable)		Good for dev/test and low traffic apps. Example: t3.micro
m Classes (General Purpose)		Balanced compute, memory and network. Example: db.m5.large
r/x Classes (Memory Optimized)		High memory workloads like Reporting, Analytics. Example: db.r5.xlarge

2. STORAGE

RDS uses EBS storage under the hood.

Storage Types

- General Purpose SSD (gp3)
- Balanced price and performance.
 - Default for most databases.



- Provisioned IOPS SSD (io1 / io2)
- High performance for
 - I/O intensive workloads.
 - You can set IOPS as per need.



Storage is...

- ☒ Scalable (increase storage without downtime)
- ☒ Encrypted at rest
- ☒ Automatically backed up
- ☒ Redundant within an Availability Zone



Key Points

- ☆ Choose the right instance class for your workload to balance performance and cost.
- ☆ Start small and scale up when your application grows.
- ☆ Use gp3 for most use cases.
- ☆ Use io1 / io2 for high I/O performance needs.
- ☆ RDS handles storage management for you.



In Simple Words,

You just focus on your application.

RDS provides the compute and storage you need.

RDS NETWORKING

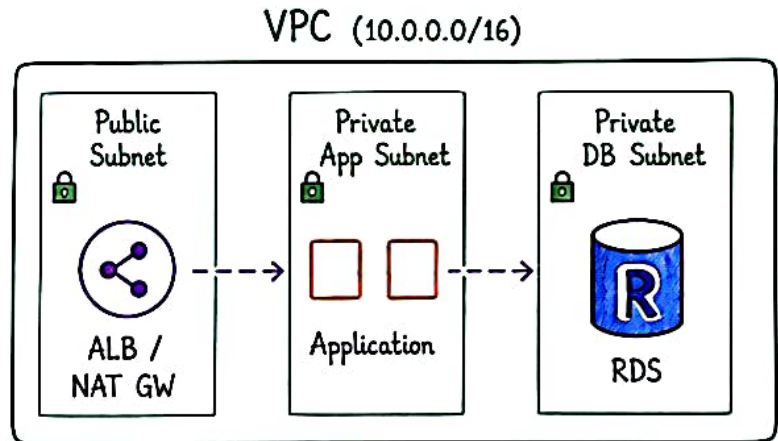
7

Networking in RDS decides how your database is accessed securely from your application.

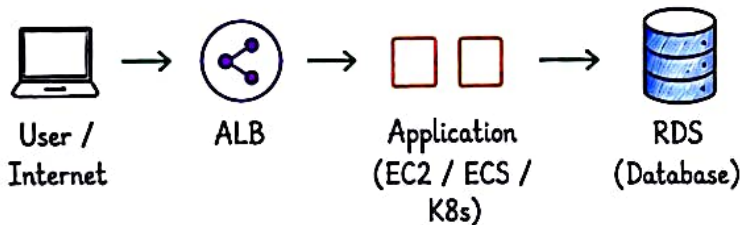
1. RDS in a VPC

RDS is launched inside your VPC in private subnets.

It is not assigned a public IP.



2. Connectivity Flow



Only resources inside the VPC (or connected via VPN / Direct Connect) can access RDS.

3. RDS Endpoint

RDS provides a private endpoint (DNS name) to connect.

Example Endpoint

mydb.123456789012.ap-south-1.rds.amazonaws.com

- Use this endpoint in your application connection string.
- The endpoint resolves to the private IP of the database.

4. Network Requirements

- Your application and RDS must be in the same VPC.
- They can be in different subnets.
- Security Groups must allow DB traffic.
- If accessing from on-premises, use VPN or Direct Connect.



5. Security Groups

RDS is controlled by Security Groups.

Allow inbound traffic on DB port (e.g., 3306 for MySQL).

Example Rule

Type : MySQL/Aurora
Port : 3306
Source : App Security Group
(not 0.0.0.0/0)



Key Takeaways

- ✓ RDS runs in your VPC private subnets.
- ✓ No public IP by default → More secure.
- ✓ Access it using private endpoint from your application.
- ✓ Control access using Security Groups.
- ✓ Use VPN / Direct Connect for on-prem access.

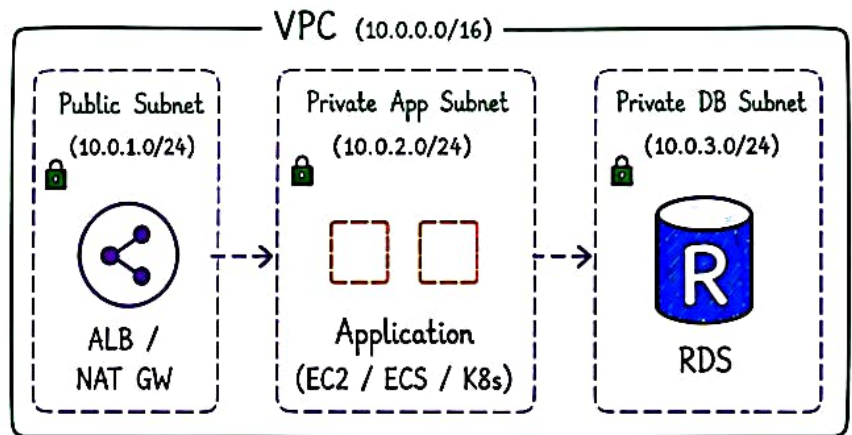
RDS SECURITY

8

RDS provides multiple layers of security to keep your database safe, private, and compliant.

1. Network Isolation

- RDS is deployed in your VPC inside **private subnets**.
- No **direct** internet access.
- Access only from allowed resources (EC2, ECS, K8s, etc.) in the same VPC.



2. Security Groups

Security Groups act as a firewall for your RDS instance.

- Allow inbound traffic only on DB port (e.g., 3306 for MySQL).
- Restrict source to specific SGs (App SG, CI/CD SG, etc.).
- Deny all other traffic by default.



Example SG Rule

Type : MySQL/Aurora
Port : 3306
Source : sg-0abc123def456...
(App Security Group)

App / EC2 / K8s SG
sg-0abc123def456



Allow 3306



RDS SG
sg-0def789 ghi012



3. Encryption



- Encrypt data at rest using AWS KMS.
- Encrypt data in transit using SSL/TLS.
- Encryption helps protect your data from unauthorized access.

4. Authentication

- Use **strong** master username and password.
- Enable IAM Database Authentication (for supported engines).
- Avoid hardcoding credentials in your application.



5. Secrets Management

Store database credentials securely using **AWS Secrets Manager**.

- Rotate secrets regularly.
- No hardcoded passwords.



6. Monitoring & Audit

- Enable RDS Logs (Error, General, Slow Query, etc.).
- Monitor using CloudWatch.
- Track who accessed and what changes were made.



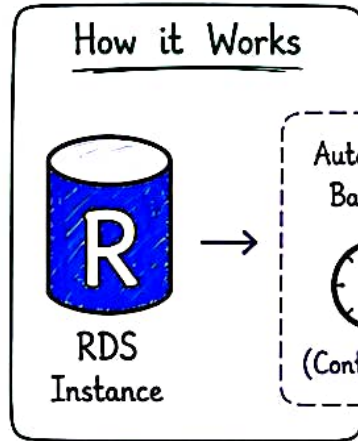
Key Takeaways

- ✓ Keep RDS in private subnets.
- ✓ Use Security Groups to control access.
- ✓ Encrypt data at rest and in transit.
- ✓ Use IAM authentication when possible.
- ✓ Store credentials in **Secrets Manager**.
- ✓ Monitor and audit for better security.

RDS BACKUPS

9

RDS automatically backs up your database so you can restore it if something goes wrong.



Restore Options



Point-in-Time Restore
Restore to any point in the backup window.



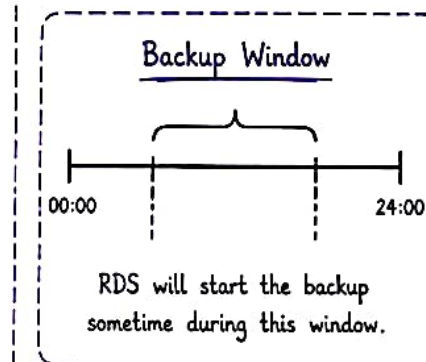
Restore to New Instance
Create a new DB instance from a backup.



Restore DB Snapshot
Restore from a manual snapshot.

1. Automated Backups

- RDS takes automatic backups of your DB.
- Backups are stored in Amazon S3.
- You can restore your DB to any point within the backup retention period.
- Default retention period: 7 days
- You can set it from 1 to 35 days.



Notes:

- Backup window duration is minimum 30 minutes.
- During backup, there is minimal performance impact.
- Backups are incremental, so they use less storage.

2. Manual Snapshots

- You can create manual snapshots anytime.
- Snapshots are stored until you delete them.
- Useful before major changes or updates.



Note:

Manual snapshots are not deleted automatically.

3. Backup Retention vs Snapshot

Automated Backups	Manual Snapshots
• Deleted automatically after retention period. • Used for point-in-time restore. • Lower storage cost.	• Stored until you delete. • Used for long-term backup or migration. • Higher storage cost.



Key Takeaways

- ✓ RDS automatically backs up your DB within the backup window.
- ✓ You can restore to any point within the retention period.
- ✓ Use manual snapshots for long-term or before important changes.
- ✓ Store backups securely in S3 with encryption.



RDS Multi-AZ

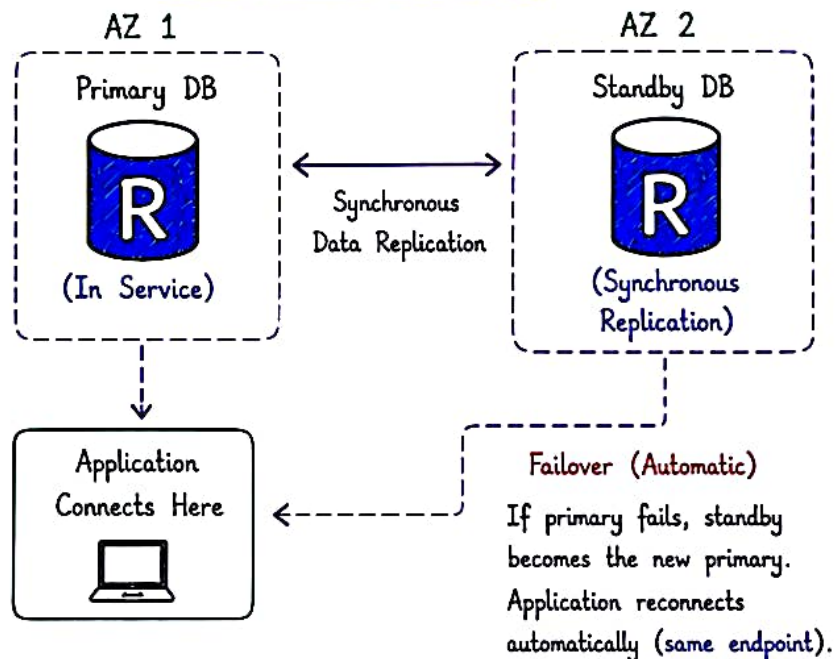
10

RDS Multi-AZ deployment increases availability and protects your database from infrastructure failure.

Why Multi-AZ?

- If the primary DB instance fails, RDS automatically fails over to the standby.
- Minimizes downtime.
- Helps meet high availability needs.

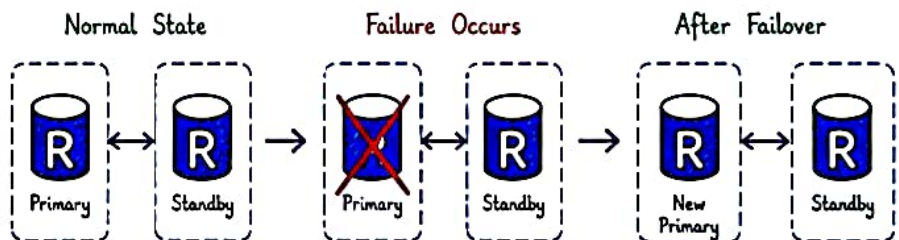
How Multi-AZ Works



Multi-AZ Benefits

- ✓ High Availability
- ✓ Automatic failover
- ✓ Data is synchronized (no data loss in most cases)
- ✓ RDS manages replication and failover for you
- ✓ No manual intervention required

Failover Example



- Failover is automatic and usually takes 60-120 seconds.
- The RDS endpoint remains the same.
- Your application reconnects automatically.

Supported Engines

- MySQL
- PostgreSQL
- MariaDB
- Oracle
- SQL Server
- Amazon Aurora

Cost



- You pay for the primary instance.
- Standby instance cost is included (no extra charge).
- Small additional storage may be used.

Key Takeaways

- ★ Multi-AZ = High Availability
- ★ RDS replicates data synchronously to standby.
- ★ Automatic failover with minimal downtime.
- ★ Same endpoint, no app changes needed.
- ★ RDS handles everything for you.

RDS READ REPLICAS

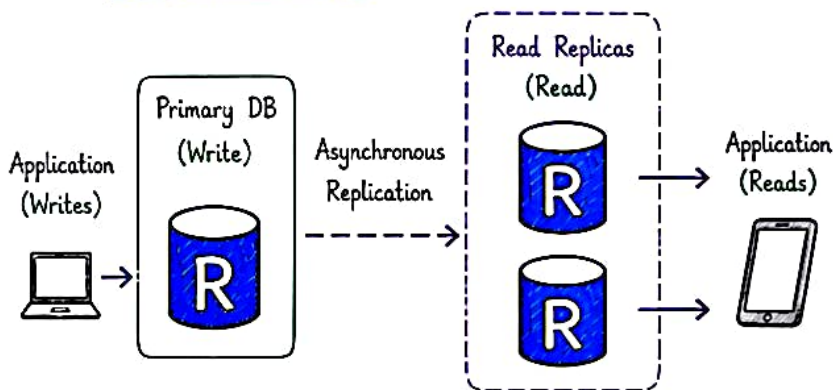
11

RDS Read Replicas let you scale read performance by creating copies of your database that handle **read-only** traffic.

1. What is Read Replica?

- A Read Replica is an exact copy of your primary DB instance.
- It is asynchronous (slight delay in replication).
- Used to scale **READ** operations without affecting the primary.
- Replication is **one-way**:
Primary → Replica (s).

2. How it Works



- Writes go to Primary.
- Reads can be distributed across one or more Read Replicas.
- If Primary fails, a Replica can be promoted to become Primary.

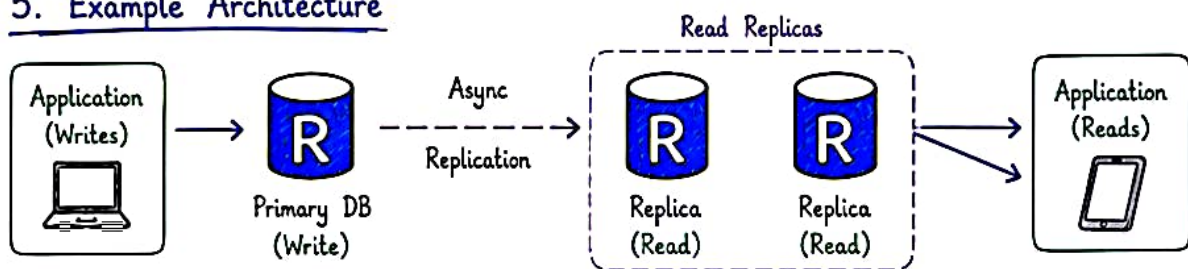
3. Use Cases

- Reporting and Analytics
- Heavy **read** workloads
- Offloading read traffic from primary
- Improving application performance

4. Key Points

- Read Replicas are in the same Region as the primary.
- You can have up to **15 Read Replicas** per DB instance.
- Replication is asynchronous (not real-time).
- Schema and data are identical to the primary.
- Replica is **read-only** (cannot accept writes).

5. Example Architecture



Things to Remember

- ★ Read Replicas improve read scaling.
- ★ They do not improve **write performance**.
- ★ Slight replication delay is expected.
- ★ Replicas can be promoted during failover.
- ★ Monitor **replication lag**.



Limitations

- ⊗ Can't write to replicas.
- ⊗ Higher cost (each replica is charged).
- ⊗ Not for cross-region scaling (use Global Database).

In Simple Words

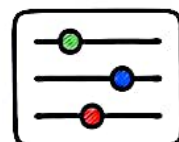
Read Replicas are extra copies of your database that handle read traffic, so your main database can focus only on **writes**. This makes your application faster and more scalable.



RDS PARAMETERS

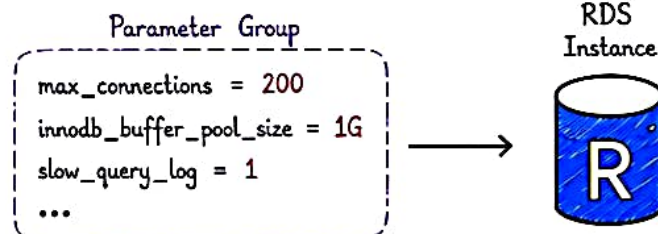
12

Parameters control the behavior of your RDS database engine.
You can modify many settings **without** changing the application.



1. What are Parameter Groups?

- A Parameter Group is a collection of database parameters.
- You associate a parameter group with an RDS instance.
- RDS uses the parameter values from that group.



2. DB Parameters

- Parameters are specific settings for the database engine.

- Examples:

- max_connections
- innodb_buffer_pool_size
- query_cache_size
- log_bin (MySQL)
- work_mem (PostgreSQL)
- shared_buffers (PostgreSQL)



Each engine (MySQL, PostgreSQL, MariaDB, etc.) has its own set of parameters.

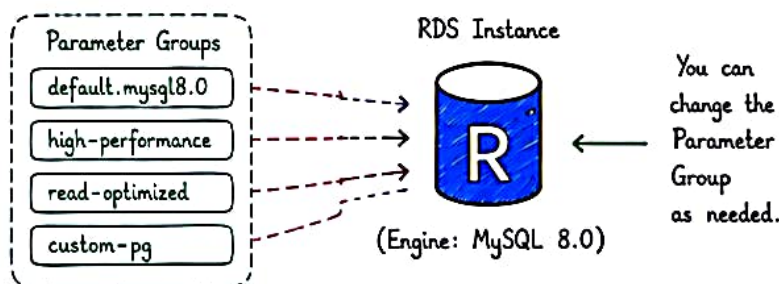
3. Dynamic vs Static Parameters

- Dynamic Parameters → Can be changed without reboot.
- Static Parameters → Require a **reboot** to take effect.

Dynamic (No Reboot)	Static (Need Reboot)
max_connections query_cache_size binlog_format time_zone log_bin_trust_function_creators ...	innodb_buffer_pool_size innodb_log_file_size character_set_server port lower_case_table_names ...

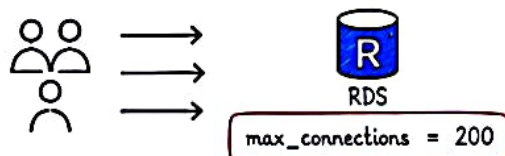
4. Parameter Group Association

- You can associate a parameter group when creating or modifying an RDS instance.
- You can switch parameter groups at any time.
- Changes apply at once (dynamic) or after reboot (static).



5. Example: max_connections

- Controls how many users can connect to the database at the same time.



- If **too low** → users get connection error.
- If **too high** → may cause memory pressure.



Always set the value based on your application load and instance size.

6. When Do Parameter Changes Require Reboot?

- If a parameter is **static** → **reboot** is required.
- If a parameter is **dynamic** → no reboot.
- You can see the "Apply Type" in the parameter list.

Apply Type	Description	Reboot Needed?
dynamic	Changes apply immediately	No
dynamic-reboot	Changes apply after reboot	Yes
static	Changes apply after reboot	Yes

RDS MONITORING

13

Monitoring helps you keep an eye on your RDS instance health, performance, and availability so you can take action before issues impact your application.



1. What is RDS Monitoring?

- Monitoring collects metrics and logs from your RDS instance.
- It helps you understand how your DB is performing.
- You can detect issues early and avoid downtime.

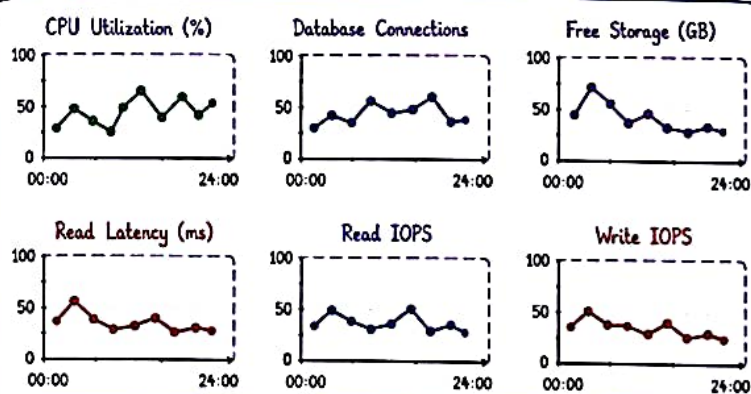
2. What Can We Monitor?

 Pro-RB				
Performance CPU, Memory, Disk, IOPS, Connections	Availability Up/Down Status, Failovers	Storage Used Storage, Free Storage, Disk Queue	Errors DB Errors, Slow Queries, Deadlocks	Throughput Read/Write Throughput, Network

3. CloudWatch Metrics for RDS

- CPUUtilization - CPU usage %
- DatabaseConnections - Active connections
- FreeStorageSpace - Free storage in GB
- ReadLatency / WriteLatency - Latency in ms
- ReadIOPS / WriteIOPS - I/O operations per sec
- SwapUsage - Swap usage in MB
- FreeableMemory - Available memory in MB
- BinLogDiskUsage - For MySQL

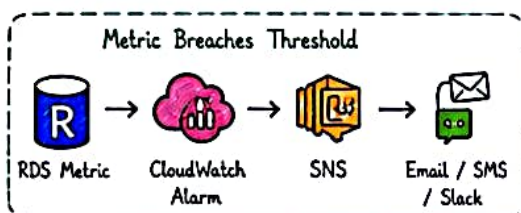
4. Example - CloudWatch Dashboard



Use CloudWatch Dashboard to visualize and monitor key metrics in real-time.

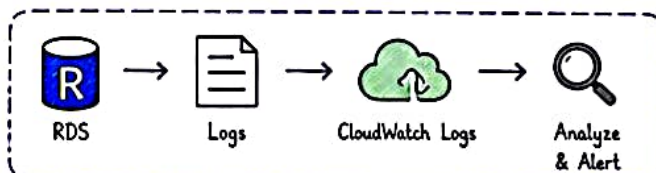
5. Alarms - Get Notified

- Create CloudWatch Alarms on important metrics.
- Get notified via SNS (Email, SMS, Slack, etc.).
- Act before small issues become big problems.



6. Logs Monitoring

- Enable Enhanced Monitoring to get OS-level metrics.
- Monitor Slow Query Logs to find performance bottlenecks.
- Monitor Error Logs to detect database issues.
- Logs can be sent to CloudWatch Logs for analysis.



★ Best Practices

- ✓ Monitor critical metrics regularly.
- ✓ Set meaningful alarm thresholds.
- ✓ Review logs and slow queries.
- ✓ Keep enough free storage space.
- ✓ Test alarms to make sure notifications work.



★ Key Takeaways

- ✓ Monitoring ensures your RDS instance is healthy and performant.
- ✓ Use CloudWatch metrics, alarms, and logs.
- ✓ Get notified early and take action.
- ✓ Proactive monitoring = Less downtime and happy users! 😊



RDS SCALING & COST

14

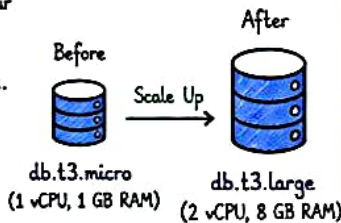


Scaling helps your application handle more load.

Cost optimization helps you pay only for what you use.

1. Vertical Scaling (Scale Up)

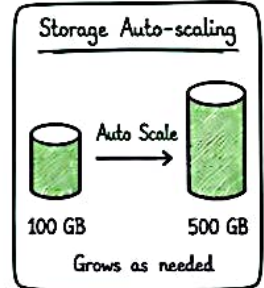
- Increase the power of your RDS instance.
- More CPU, Memory, Network.
- Good for handling more traffic or heavy queries.
- Requires a reboot during modification.



Choose the right instance class based on your workload.

2. Storage Scaling

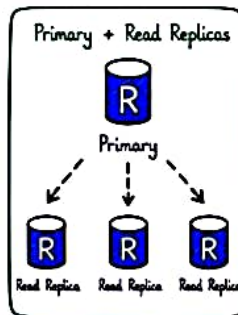
- Increase your allocated storage.
- RDS supports storage auto-scaling (up to 64 TB for many engines).
- Helps when your database grows over time.
- Storage scaling is applied immediately (no reboot).



Enable Auto-scaling to avoid "disk full" issues.

3. Read Replicas (Read Scaling)

- Create read replicas to distribute read traffic.
- Improves read performance.
- Replicas are asynchronous by default.
- Can promote replica to standalone if needed.



Use Read Replicas for reporting, analytics and read-heavy workloads.

4. RDS Pricing Overview

Component	What You Pay For
Instance	Instance type and size (vCPU, Memory, Network)
Storage	GB per month (general purpose SSD) + provisioned IOPS (if used)
I/O Requests	Read/Write IOPS beyond the free tier
Backups	Automated backups storage beyond free allowance
Data Transfer	Data transfer out to Internet or cross-region

5. Reserved Instances (Save Cost)

- Commit for 1 or 3 years to get discount (up to ~60%).
- Good for stable and predictable workloads.
- Convertible RIs give more flexibility.

RI Benefits

- ☒ Lower cost ↓
- ☒ Budget predictable
- ☒ Long term savings
- ☒ Multiple payment options

6. Cost Optimization Tips

- ✓ Right-size your instance.
- ✓ Use Storage Auto-scaling.
- ✓ Use Read Replicas for read-heavy workloads.
- ✓ Stop non-production databases when not in use.
- ✓ Use Reserved Instances for long term.
- ✓ Delete old backups & snapshots.



7. When to Use What?

Use Vertical Scaling

When:

- Your app is slow due to high CPU, memory or network usage.
- You need more power.



Use Storage Scaling

When:

- You are running out of storage.
- Your data is growing continuously.



Use Read Replicas

When:

- Your app has heavy read traffic.
- You want to offload reads from primary.



Key Takeaways



- ✓ Scale up (vertical) for more power.
- ✓ Scale storage as your data grows.
- ✓ Use read replicas for read scaling.
- ✓ Understand each cost component.
- ✓ Optimize to save money and improve performance.



Remember

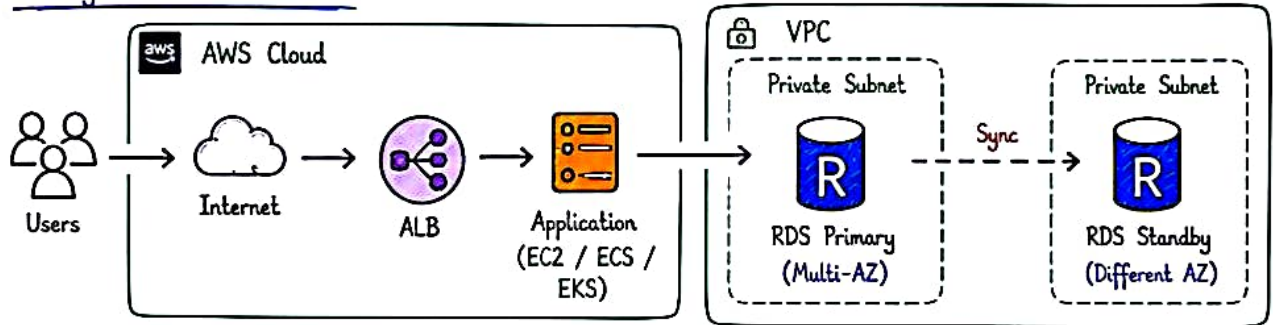
- More traffic → Scale instance or add replicas.
- More data → Scale storage.
- High cost → Optimize and right-size.
- Plan ahead to balance performance and cost.



✓ Scaling + Cost Optimization = Better Performance + Lower Cost 😊

RDS Architecture is designed for high availability, security, scalability and ease of management.

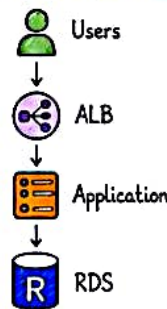
1. High Level Architecture



2. Key Components

-  **VPC** : Isolated network for secure communication.
-  **Private Subnets** : RDS runs in private subnets (not reachable from Internet).
-  **Security Groups** : Control inbound and outbound traffic to RDS.
-  **RDS Multi-AZ** : Primary and standby in different AZs for high availability.
-  **Secrets Manager** : Stores DB credentials securely.
-  **CloudWatch** : Monitoring, metrics, alarms and logs.

3. Security Flow



- Users access the application via ALB.
- Application connects to RDS using private endpoint inside VPC.
- Security Group allows traffic only from application to RDS.
- RDS is not accessible from Internet.

4. Security Groups

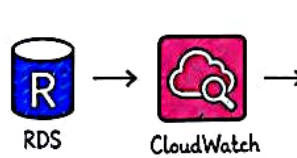
Security Group	Inbound Rules (Example)	Outbound Rules
ALB-SG	80 (HTTP) from 0.0.0.0/0 443 (HTTPS) from 0.0.0.0/0	All to Web-SG
Web-SG	80/443 from ALB-SG	3306 (MySQL) to RDS-SG
RDS-SG	3306 (MySQL) from Web-SG	All (Default)

5. Secrets Manager Integration



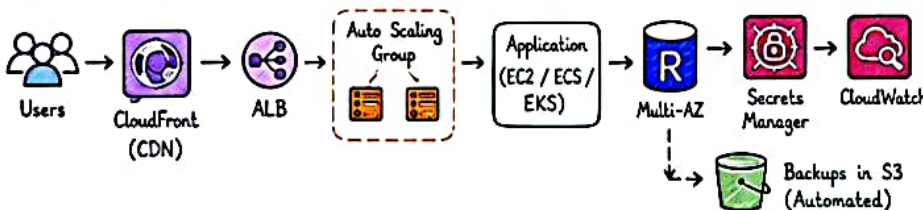
- 💡 Application retrieves DB credentials securely at runtime. No hardcoded credentials.

6. Monitoring with CloudWatch



- **Metrics**
CPUUtilization
DatabaseConnections
FreeStorageSpace
ReadLatency, WriteLatency
- **Alarms**
- **Logs** (Error, Slow Query, Audit)

7. Production Architecture Example (Real World)



- ✓ High Availability (Multi-AZ)
- ✓ Secure (Private Subnet + SG)
- ✓ Scalable (ASG + Read Replicas)
- ✓ Monitored (CloudWatch)
- ✓ Backed up (Automated)
- ✓ Cost optimized

★ Key Takeaways

- ① RDS runs in private subnets for security.
- ② Multi-AZ provides high availability.
- ③ Security Groups control access to RDS.
- ④ Secrets Manager keeps credentials safe.
- ⑤ CloudWatch helps you monitor and alert.
- ⑥ Follow best practices for production ready architecture.

✓ Best Practice

- Use Multi-AZ for production.
- Enable automated backups.
- Use IAM & Secrets Manager.
- Monitor and set alarms.
- Use Read Replicas for read scaling.
- Cost optimized



💡 A well designed RDS architecture = High Availability + Security + Scalability + Cost Efficiency 😊

RDS TROUBLESHOOTING

16



When something goes wrong in RDS, follow a systematic approach to find the root cause and fix it quickly.



App



Network



RDS

1. Connection Problems

- Check the endpoint (correct and reachable).
- Verify the port (default 3306 for MySQL, 5432 for PostgreSQL).
- Check Security Group inbound rules.
- Ensure the RDS is in the same VPC / subnet or routing is correct.
- Check NACL rules.
- Check RDS status (Available or not).

2. Authentication Problems

- Verify username and password.
- Check the database name.
- For IAM authentication, check IAM role and permissions.
- Check Secrets Manager for correct credentials and rotation.
- Ensure the user has proper privileges.

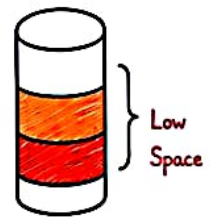


3. Performance Problems

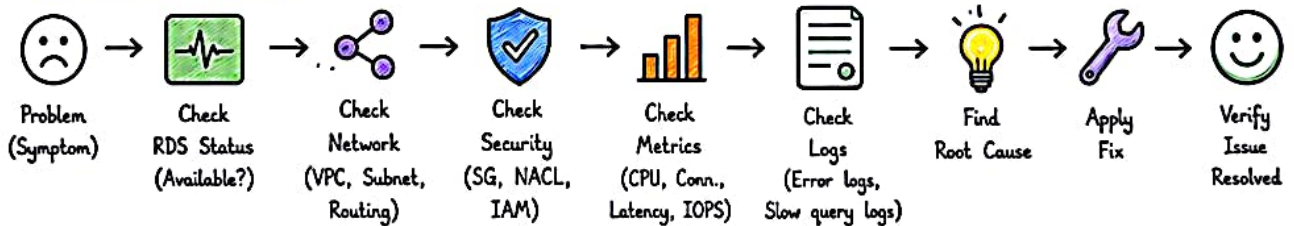
- Check CPU Utilization in CloudWatch.
- Check DatabaseConnections (too many connections?).
- Check ReadLatency / WriteLatency.
- Check FreeStorageSpace and IOPS.
- Look for slow queries using Performance Insights.
- Consider scaling (vertical or read replicas).

4. Storage Problems

- Check FreeStorageSpace metric.
- If low, enable Storage Auto Scaling.
- Review large tables, logs, or unused data.
- Delete old backups and snapshots if not needed.
- Consider moving data to S3 if needed.



Troubleshooting Flow (Follow this order)



Common Issues & Quick Checks

- Can't connect to RDS → Check endpoint, port, SG, subnet, route table.
- AccessDenied / 403 → Check IAM roles, SG rules, user privileges.
- Login failed → Check username, password, DB name, auth method.
- High CPU → Check running queries, connections, instance size.
- Too many connections → Check max_connections, connection pool.
- Slow queries → Check Performance Insights, optimize query.
- Database full → Check FreeStorageSpace, clean old data.
- Backup issues → Check backup window, storage, permissions.
- Replication lag (Read Replica) → Check network, load, replica health, large transactions.



Interview Questions

1. RDS connection timeout - what will you check?
2. You get AccessDenied while connecting - what could be the reason?
3. CPU is very high on RDS - how will you troubleshoot it?
4. Database storage is full - what steps will you take?
5. Application cannot connect to RDS after creation - what could be wrong?



Tip:

Always start with the basics (status, network, security) before digging into performance or logs.

